

MODUL VI - INTERRUPT DAN TIMER

Nayla Zazki Kirani (H1H024007)

Asisten: Arga Aryanta Indrafata

Tanggal Percobaan: 12/05/2026

TK244005-Praktikum Sistem Mikrokontroler

Laboratorium Baru – Jurusan Informatika UNSOED

Abstrak

Praktikum Modul 6 membahas penerapan interrupt dan timer pada mikrokontroler Arduino Uno. Percobaan 6A difokuskan pada penggunaan external interrupt menggunakan push button untuk mengendalikan LED tanpa metode polling. Sementara itu, Percobaan 6B menerapkan timer berbasis fungsi millis() untuk menghasilkan LED blinking secara non-blocking. Berdasarkan hasil pengujian, mekanisme interrupt mampu memberikan respons yang cepat terhadap input eksternal dengan penggunaan sumber daya yang lebih efisien dibandingkan polling. Selain itu, penggunaan millis() memungkinkan program tetap menjalankan proses lain tanpa menghentikan eksekusi utama. Implementasi kedua metode ini menjadi dasar penting dalam pengembangan sistem embedded yang responsif dan efisien.

Kata kunci: Arduino Uno, external interrupt, ISR, millis(), timer, non-blocking, sistem tertanam.

1. PENDAHULUAN

Sistem embedded merupakan sistem komputasi yang dirancang untuk menjalankan fungsi tertentu secara spesifik dan real-time. Dalam pemrograman mikrokontroler, salah satu permasalahan utama adalah bagaimana sistem dapat merespons suatu kejadian eksternal dengan cepat tanpa mengurangi efisiensi proses utama. Pendekatan polling yang umum digunakan mengharuskan mikrokontroler memeriksa kondisi input secara terus-menerus sehingga menyebabkan pemborosan sumber daya dan meningkatkan waktu respons sistem [1].

Untuk mengatasi permasalahan tersebut, digunakan mekanisme interrupt dan timer. Interrupt memungkinkan mikrokontroler menangani suatu event secara langsung tanpa harus melakukan pengecekan berulang, sedangkan timer memungkinkan pengelolaan waktu yang lebih akurat tanpa menghentikan jalannya program utama [2]. Kedua konsep ini banyak diterapkan pada sistem IoT, robotika, monitoring, maupun otomasi industri yang membutuhkan respons cepat dan pengaturan waktu presisi [3].

Praktikum ini bertujuan untuk memahami implementasi external interrupt pada Arduino Uno menggunakan push button dan LED pada Percobaan 6A, serta memahami penggunaan timer non-blocking berbasis millis() pada Percobaan 6B. Melalui praktikum ini diharapkan mahasiswa dapat memahami konsep ISR, variabel volatile, interrupt, dan multitasking sederhana pada sistem embedded [4].

2. STUDI PUSTAKA

2.1 INTERRUPT PADA MIKROKONTROLER

Interrupt adalah mekanisme perangkat keras yang memungkinkan mikrokontroler menghentikan sementara eksekusi program utama untuk menangani suatu event tertentu melalui fungsi khusus yang disebut Interrupt Service Routine (ISR). Ketika sebuah interrupt terjadi, mikrokontroler menyimpan konteks program saat ini, menjalankan ISR, lalu melanjutkan program utama dari titik yang sama [1]. Pada Arduino Uno (berbasis ATmega328P), external interrupt tersedia pada pin 2 (INT0) dan pin 3 (INT1), dengan empat mode pemicu: RISING, FALLING, CHANGE, dan LOW [2].

Menurut Mardiyanto et al. [3], penggunaan interrupt pada sistem embedded mampu mengurangi latensi respons secara signifikan dibandingkan metode polling, terutama pada aplikasi yang membutuhkan deteksi event presisi tinggi seperti encoder rotary dan sensor ultrasonik. ISR sebaiknya dirancang sesingkat mungkin karena selama ISR berjalan, proses utama terhenti sementara. Fungsi `delay()` dan `Serial.print()` tidak disarankan digunakan di dalam ISR karena bergantung pada interrupt internal Arduino [2].

2.2 VARIABEL VOLATILE

Variabel yang digunakan bersama antara ISR dan program utama harus dideklarasikan dengan keyword `volatile`. Keyword ini memberitahu compiler bahwa nilai variabel dapat berubah sewaktu-waktu di luar alur normal program, sehingga compiler tidak akan melakukan optimasi yang dapat menyebabkan nilai variabel tidak terbaca dengan benar [2]. Tanpa `volatile`, compiler dapat menyimpan nilai variabel di register CPU dan tidak memeriksa perubahan dari ISR, yang berpotensi menyebabkan perilaku sistem yang tidak terduga [1].

2.3 TIMER DAN FUNGSI `MILLIS()`

Timer merupakan modul internal mikrokontroler yang digunakan untuk menghitung waktu berdasarkan clock sistem. Arduino Uno memiliki tiga modul timer: Timer0 (8-bit), Timer1 (16-bit), dan Timer2 (8-bit). Fungsi `millis()` memanfaatkan Timer0 untuk mengembalikan waktu dalam milidetik sejak Arduino dinyalakan [2]. Berbeda dengan fungsi `delay()` yang bersifat blocking (menghentikan seluruh eksekusi program), `millis()` bersifat non-blocking sehingga program tetap dapat menjalankan tugas-tugas lain secara bersamaan [4].

Prasetyo dan Kusuma [5] menyatakan bahwa penggunaan `millis()` dalam implementasi multitasking kooperatif pada Arduino terbukti efektif untuk mengelola beberapa tugas periodik secara simultan tanpa menggunakan sistem operasi real-time (RTOS). Pola pemrograman ini sangat cocok untuk sistem IoT sederhana yang membutuhkan pemantauan beberapa sensor atau aktuator secara bersamaan dengan konsumsi memori yang minimal [3].

3. METODOLOGI

3.1 ALAT DAN BAHAN

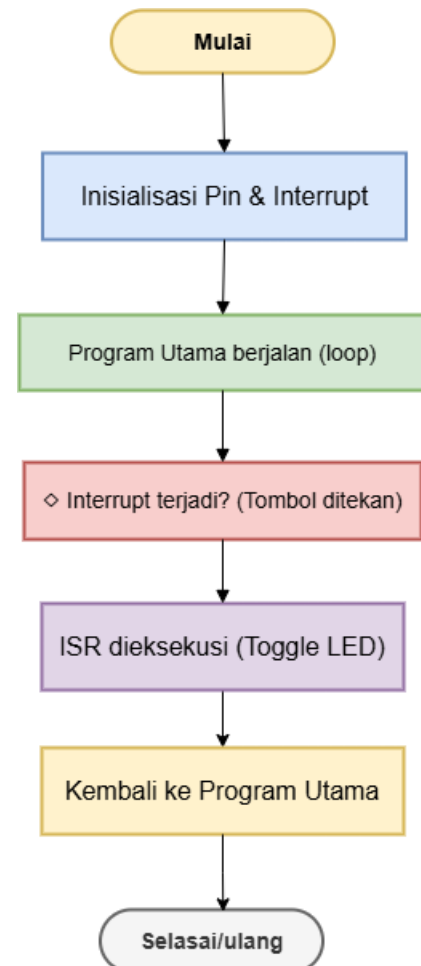
Berikut adalah alat dan bahan yang digunakan dalam kedua percobaan pada modul ini:

- Arduino Uno (mikrokontroler ATmega328P)
- Push Button
- LED sebanyak 2 buah
- Resistor 220Ω (untuk LED)
- Resistor 10kΩ (pull-down untuk push button)
- Kabel Jumper

- Breadboard
- Komputer dengan Arduino IDE / PlatformIO

3.2 PERCOBAAN 6A: EXTERNAL INTERRUPT

Diagram alur berikut menggambarkan cara kerja sistem pada Percobaan 6A menggunakan mekanisme external interrupt:



Gambar 3.1. Diagram Alur Percobaan 6A – External Interrupt

Konfigurasi Rangkaian

Tabel 3.1 Wiring Komponen Percobaan 6A

No	Komponen	Pin Arduino
1	LED	Pin 13
2	Push Button	Pin 2 (INT0)
3	GND	GND
4	VCC	5V

Langkah Percobaan

1. Menyiapkan alat dan bahan serta merangkai komponen pada breadboard sesuai Tabel 1.
2. Membuat sketch Arduino dengan nama modul6_interrupt menggunakan PlatformIO / Arduino IDE.
3. Mendeklarasikan variabel volatile bool ledState untuk digunakan bersama antara ISR dan program utama.
4. Mendaftarkan ISR `tombolInterrupt()` menggunakan `attachInterrupt()` pada pin 2 dengan mode FALLING.
5. Meng-compile dan meng-upload program ke board Arduino Uno.
6. Mengamati perubahan kondisi LED ketika tombol ditekan dan memberikan analisis.

Berikut kode program inti Percobaan 6A: External Interrupt:

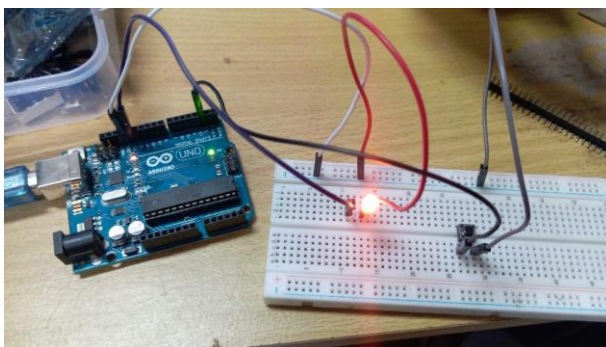
```
#include <Arduino.h>

volatile bool ledState = false;
void tombolInterrupt() {
  ledState = !ledState;
}

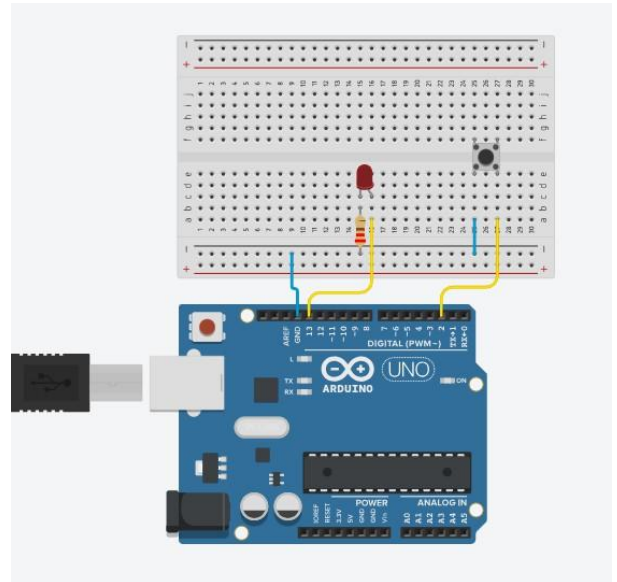
void setup() {
  pinMode(13, OUTPUT);
  pinMode(2, INPUT_PULLUP);

  attachInterrupt(
    digitalPinToInterrupt(2),
    tombolInterrupt,
    FALLING
  );
}

void loop() {
  digitalWrite(13, ledState);
}
```



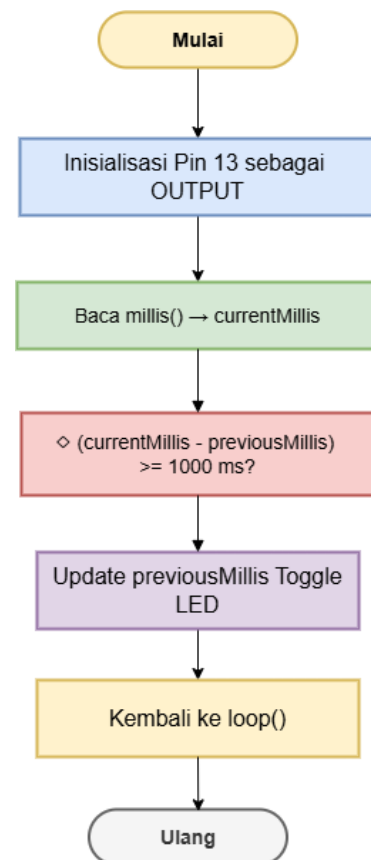
Gambar 3.2 Percobaan 6A: External Interrupt



Gambar 3.3 Wiring Percobaan 6A

3.3 PERCOBAAN 6B: TIMER MILLIS()

Diagram alur berikut menggambarkan cara kerja sistem pada Percobaan 6B menggunakan fungsi `millis()` sebagai timer non-blocking:



Gambar 3.4 Diagram Alur Percobaan 6B – Timer menggunakan millis()

Konfigurasi Rangkaian

Tabel 3.2 Wiring Komponen Percobaan 6B

No	Komponen	Pin Arduino
1	LED	Pin 13
2	GND	GND

Langkah Percobaan

1. Menyiapkan alat dan bahan serta merangkai komponen LED pada breadboard sesuai Tabel 2.
2. Membuat sketch Arduino dengan nama modul6_timer.
3. Mendeklarasikan variabel `previousMillis`, `interval = 1000`, dan `ledState` untuk pengelolaan timer.
4. Menggunakan kondisi `(millis() - previousMillis >= interval)` pada `loop()` untuk menggantikan `delay()`.
5. Meng-compile dan meng-upload program ke board Arduino Uno.
6. Mengamati perilaku LED yang berkedip setiap 1 detik dan memverifikasi sistem tetap responsif

Berikut kode program inti Percobaan 6B: Timer Milles():

```
#include <Arduino.h>

unsigned long previousMillis = 0;
const long interval = 1000;
bool ledState = false;

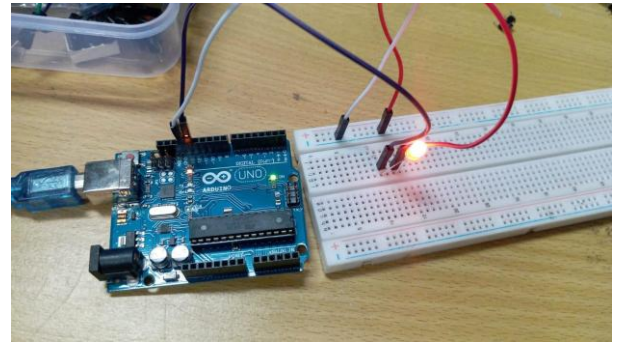
void setup() {
  pinMode(13, OUTPUT);
}

void loop() {

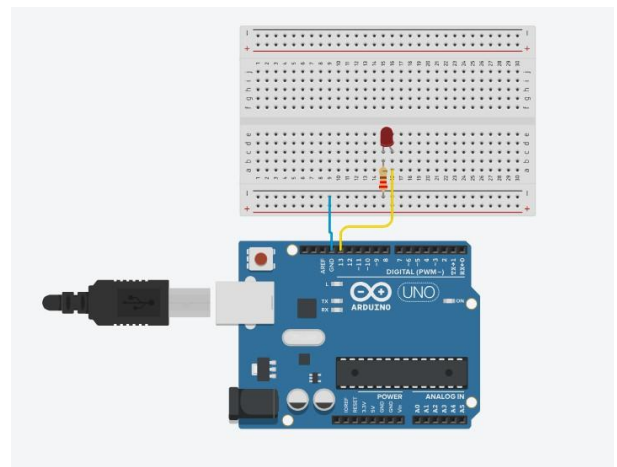
  unsigned long currentMillis =
  millis();

  if (currentMillis - previousMillis
  >= interval) {

    previousMillis = currentMillis;
    ledState = !ledState;
    digitalWrite(13, ledState);
  }
}
```



Gambar 3.5 Percobaan 6B:Timer menggunakan millis()



Gambar 3.6 wiring percobaan 6B

4. HASIL DAN ANALISIS

4.1 HASIL PERCOBAAN 6A: EXTERNAL INTERRUPT

Pada percobaan 6A, setelah program di-upload ke Arduino Uno, LED yang terhubung ke pin 13 berhasil dikontrol menggunakan push button melalui mekanisme interrupt. Setiap kali tombol ditekan (sisi FALLING dari sinyal), ISR `tombolInterrupt()` dipanggil secara otomatis dan variabel `ledState` di-toggle. Program utama pada fungsi `loop()` hanya bertugas menulis nilai `ledState` ke pin 13 menggunakan `digitalWrite()`, tanpa melakukan polling kondisi tombol.

Sistem berhasil merespons penekanan tombol secara instan tanpa penundaan yang signifikan, menunjukkan bahwa mekanisme interrupt bekerja sesuai spesifikasi. LED berpindah kondisi ON ke OFF atau sebaliknya setiap kali tombol ditekan, dan sistem tidak memerlukan pengecekan kondisi tombol secara terus-menerus pada loop utama.

Pertanyaan Percobaan 6A

→ Pertanyaan 1:

Jelaskan proses bagaimana tombol dapat mengubah kondisi LED menggunakan interrupt!

Pembahasan:

Pada program ini, push button dihubungkan ke pin 2 Arduino yang digunakan sebagai pin interrupt. Ketika tombol ditekan atau dilepas, Arduino akan mendeteksi perubahan sinyal pada pin tersebut sesuai mode interrupt yang digunakan.

Saat interrupt terjadi, program utama akan dihentikan sementara dan Arduino menjalankan fungsi ISR (*Interrupt Service Routine*) yaitu:

```
void tombolInterrupt() {  
    ledState = !ledState;  
}
```

Pada fungsi tersebut, variabel `ledState` diubah menjadi kebalikan dari kondisi sebelumnya. Jika sebelumnya LED dalam kondisi mati, maka LED akan menyala, dan sebaliknya.

Setelah ISR selesai dijalankan, program utama kembali berjalan dan nilai `ledState` digunakan untuk mengatur kondisi LED melalui:

```
digitalWrite(13, ledState);
```

Dengan mekanisme interrupt, Arduino tidak perlu terus-menerus memeriksa kondisi tombol menggunakan polling sehingga sistem menjadi lebih cepat dan efisien.

→ Pertanyaan 2:

Apa fungsi `attachInterrupt()` pada program tersebut?

Pembahasan:

Fungsi `attachInterrupt()` digunakan untuk menghubungkan pin interrupt dengan fungsi ISR yang akan dijalankan ketika kondisi tertentu terjadi.

Pada program digunakan:

```
attachInterrupt(  
    digitalPinToInterrupt(2),  
    tombolInterrupt,  
    RISING  
);
```

Penjelasannya adalah sebagai berikut:

- `digitalPinToInterrupt(2)` digunakan untuk menentukan pin interrupt yaitu pin 2.
- `tombolInterrupt` merupakan fungsi ISR yang akan dijalankan saat interrupt terjadi.
- `RISING` merupakan mode interrupt yang digunakan sebagai pemicu interrupt.

Dengan fungsi ini, Arduino dapat secara otomatis menjalankan ISR ketika terjadi perubahan sinyal sesuai mode yang dipilih.

→ Pertanyaan 3:

Mengapa pada ISR tidak disarankan menggunakan `delay()` dan `Serial.print()`?

Pembahasan:

Pada saat ISR berjalan, program utama Arduino akan berhenti sementara. Oleh karena itu ISR harus dibuat sesingkat dan secepat mungkin. Fungsi `delay()` tidak disarankan digunakan di dalam ISR karena bersifat blocking sehingga dapat membuat sistem berhenti terlalu lama dan mengganggu proses interrupt lainnya.

Selain itu, `Serial.print()` juga tidak disarankan karena proses komunikasi serial membutuhkan waktu relatif lama dan menggunakan interrupt internal. Jika digunakan di dalam ISR dapat menyebabkan program menjadi lambat, data serial rusak, atau bahkan sistem menjadi tidak stabil.

Karena itu ISR sebaiknya hanya berisi proses sederhana seperti mengubah nilai variabel.

→ Pertanyaan 4:

Apa fungsi keyword volatile pada variabel ledState?

Pembahasan:

Keyword `volatile` digunakan agar compiler selalu membaca nilai terbaru dari variabel langsung dari memori.

Pada program digunakan:

```
volatile    bool    ledState    =  
false;
```

Hal ini diperlukan karena variabel `ledState` dapat diubah baik di dalam ISR maupun di program utama (`loop`). Dengan menggunakan `volatile`, compiler tidak akan melakukan optimasi yang dapat menyebabkan perubahan nilai dari ISR tidak terbaca dengan benar oleh program utama.

→ Pertanyaan 5:

Pada percobaan digunakan mode interrupt FALLING. Modifikasikan program menggunakan mode interrupt lain (RISING, CHANGE, atau LOW)

Pembahasan:

A. MODE RISING

```
#include <Arduino.h>  
  
volatile bool ledState =  
false;  
  
// ISR  
void tombolInterrupt() {  
    ledState = !ledState;  
}  
  
void setup() {  
    pinMode(13, OUTPUT);
```

```
pinMode(2, INPUT_PULLUP);  
  
attachInterrupt(  
    digitalPinToInterrupt(2),  
    tombolInterrupt,  
    RISING  
);  
}  
  
void loop() {  
    digitalWrite(13, ledState);  
}
```

Penjelasan Program

Program menggunakan mode interrupt RISING, yaitu interrupt akan dipicu ketika sinyal berubah dari LOW ke HIGH. Pada konfigurasi `INPUT_PULLUP`, kondisi ini biasanya terjadi saat tombol dilepas. Ketika interrupt terjadi, fungsi `tombolInterrupt()` dijalankan untuk mengubah nilai `ledState`. Nilai tersebut kemudian digunakan untuk mengatur kondisi LED pada pin 13.

Analisis Perilaku LED

Pada mode RISING, LED berubah kondisi saat tombol dilepas. Akibatnya respon LED terlihat muncul setelah pengguna melepaskan tombol.

B. MODE CHANGE

```
#include <Arduino.h>  
  
volatile bool ledState =  
false;  
  
void tombolInterrupt() {  
    ledState = !ledState;  
}  
  
void setup() {  
    pinMode(13, OUTPUT);  
    pinMode(2, INPUT_PULLUP);
```

```
attachInterrupt(
  digitalPinToInterrupt(2),
  tombolInterrupt,
  CHANGE
);

void loop() {
  digitalWrite(13, ledState);
}
```

Penjelasan Program

Mode `CHANGE` akan memicu interrupt setiap terjadi perubahan sinyal, baik dari `HIGH` ke `LOW` maupun dari `LOW` ke `HIGH`. Dengan demikian fungsi ISR akan dijalankan dua kali, yaitu saat tombol ditekan dan saat tombol dilepas.

Analisis Perilaku LED

LED berubah kondisi lebih sering karena interrupt dipicu pada dua kondisi berbeda. Dalam beberapa kasus LED dapat terlihat berkedip cepat atau kembali ke kondisi awal ketika tombol ditekan dan dilepas. .

C. MODE LOW

```
#include <Arduino.h>

volatile bool ledState =
false;

void tombolInterrupt() {
  ledState = !ledState;
}

void setup() {
  pinMode(13, OUTPUT);
  pinMode(2, INPUT_PULLUP);

  attachInterrupt(
    digitalPinToInterrupt(2),
```

```
tombolInterrupt,
  LOW
);
}

void loop() {
  digitalWrite(13, ledState);
}
```

Penjelasan Program

Mode `LOW` akan terus memicu interrupt selama pin berada pada kondisi `LOW`. Pada konfigurasi `INPUT_PULLUP`, kondisi `LOW` terjadi ketika tombol ditekan. Akibatnya ISR akan dipanggil terus-menerus selama tombol ditekan.

Analisis Perilaku LED

LED dapat berkedip sangat cepat atau berubah kondisi secara tidak stabil karena fungsi ISR dijalankan berulang kali selama tombol ditekan dan ditahan.

Perbedaan Mode Interrupt

Tabel 3.3 Perbedaan Mode Interrupt

Mode Interrupt	Kondisi Pemicu	Perilaku LED
FALLING	Perubahan <code>HIGH</code> ke <code>LOW</code>	LED berubah saat tombol ditekan
RISING	Perubahan <code>LOW</code> ke <code>HIGH</code>	LED berubah saat tombol dilepas
CHANGE	Semua perubahan sinyal	LED berubah saat ditekan dan dilepas
LOW	Saat pin dalam kondisi <code>LOW</code>	LED berkedip cepat atau tidak stabil

4.2 HASIL PERCOBAAN 6B: TIMER MİLIS()

Pada percobaan 6B, LED pada pin 13 berhasil berkedip dengan periode 1 detik (500 ms `ON`, 500 ms `OFF`) menggunakan pendekatan non-blocking `millis()`. Program secara

terus-menerus membaca nilai `millis()` dan membandingkannya dengan `previousMillis`. Ketika selisih waktu mencapai atau melebihi 1000 ms, status LED di-toggle dan `previousMillis` diperbarui.

Sistem terbukti tetap responsif karena fungsi `loop()` tidak diblokir selama periode penghitungan waktu. Berbeda dengan penggunaan `delay(1000)` yang akan memblokir CPU selama satu detik penuh, pendekatan `millis()` memungkinkan eksekusi instruksi lain di dalam `loop()` tetap berjalan selama menunggu interval waktu tercapai.

Jawaban Pertanyaan Percobaan 6B

→ Pertanyaan 1:

Jelaskan bagaimana fungsi `millis()` bekerja pada program tersebut!

Pembahasan:

Fungsi `millis()` digunakan untuk menghitung waktu sejak Arduino pertama kali dinyalakan dalam satuan milidetik (ms). Pada program, fungsi ini dipakai untuk menentukan kapan LED harus berubah kondisi tanpa menghentikan jalannya program utama.

Program mengambil waktu saat ini menggunakan:

```
unsigned long currentMillis =  
millis();
```

Kemudian nilai tersebut dibandingkan dengan waktu sebelumnya yang tersimpan pada variabel `previousMillis`. Jika selisih waktunya telah mencapai interval yang ditentukan, maka status LED akan diubah dari ON menjadi OFF atau sebaliknya.

Sebagai contoh:

```
if (currentMillis -  
previousMillis >= interval)
```

Kondisi tersebut berarti LED pertama akan berubah kondisi setiap 1000 ms atau 1 detik. Metode ini memungkinkan Arduino menjalankan

beberapa proses secara bersamaan tanpa menghentikan program.

→ Pertanyaan 2:

Apa perbedaan utama antara `delay()` dan `millis()`?

Pembahasan:

Perbedaan utama antara `delay()` dan `millis()` terletak pada cara kerja program selama proses penundaan waktu berlangsung. Fungsi `delay()` bersifat blocking, artinya selama waktu `delay` berlangsung program akan berhenti sementara dan tidak dapat menjalankan proses lain. Sebagai contoh:

```
delay(11000);
```

Pada kondisi tersebut Arduino akan berhenti selama 1 detik sebelum melanjutkan program berikutnya. Sedangkan `millis()` bersifat non-blocking karena program tetap berjalan sambil menghitung waktu. Dengan metode ini Arduino tetap dapat menjalankan tugas lain seperti membaca sensor atau mengendalikan perangkat lain secara bersamaan.

→ Pertanyaan 3:

Mengapa metode `millis()` disebut non-blocking?

Pembahasan:

Metode `millis()` disebut non-blocking karena tidak menghentikan jalannya program utama saat melakukan perhitungan waktu. Program tetap terus melakukan perulangan pada fungsi `loop()` sambil mengecek apakah waktu yang ditentukan sudah tercapai.

Pengecekan dilakukan menggunakan:

```
if (currentMillis -  
previousMillis >= interval)
```

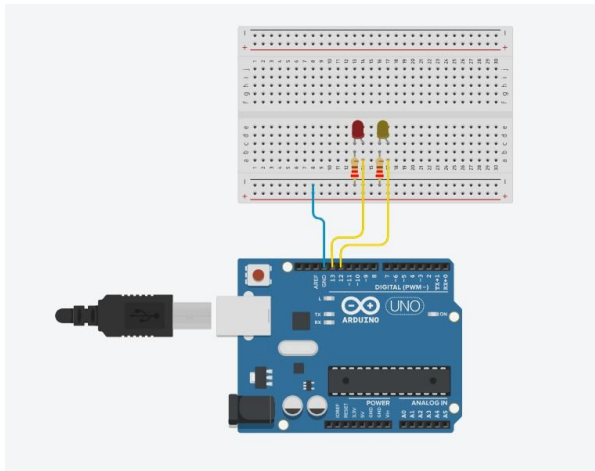
Dengan metode ini, Arduino dapat menjalankan beberapa task secara bersamaan sehingga sistem menjadi lebih

responsif dan efisien dibandingkan menggunakan `delay()`.

→ **Pertanyaan 4:**

Modifikasi program agar: LED pertama berkedip setiap 1 detik, LED kedua, berkedip setiap 500 ms, dan tanpa menggunakan `delay()`

Pembahasan:



Gambar 3.7 wiring modifikasi 6B

```
#include <Arduino.h>

// LED 1
unsigned long previousMillis1
= 0;
const long interval1 = 1000;
bool ledState1 = false;

// LED 2
unsigned long previousMillis2
= 0;
const long interval2 = 500;
bool ledState2 = false;

void setup() {
  pinMode(13, OUTPUT); // LED
  1
  pinMode(12, OUTPUT); // LED
  2
```

```
}

void loop() {
  // Waktu sekarang
  unsigned long currentMillis
= millis();

  // LED 1 berkedip tiap 1
  detik
  if (currentMillis -
  previousMillis1 >= interval1)
  {
    previousMillis1 =
    currentMillis;

    ledState1 = !ledState1;
    digitalWrite(13,
    ledState1);
  }

  // LED 2 berkedip tiap 500
  ms
  if (currentMillis -
  previousMillis2 >= interval2)
  {
    previousMillis2 =
    currentMillis;

    ledState2 = !ledState2;
    digitalWrite(12,
    ledState2);
  }
}
```

Penjelasan Program

Program diawali dengan pemanggilan library menggunakan:

```
#include <Arduino.h>
```

Library ini digunakan untuk mengakses fungsi-fungsi dasar Arduino.

Selanjutnya dibuat variabel untuk LED pertama:

```
unsigned long previousMillis1
= 0;
const long interval1 = 1000;
```

```
bool ledState1 = false;
```

Variabel `previousMillis1` digunakan untuk menyimpan waktu terakhir LED berubah kondisi. Variabel `interval1` menentukan interval kedipan LED selama 1000 ms atau 1 detik. Sedangkan `ledState1` digunakan untuk menyimpan status LED apakah ON atau OFF.

Untuk LED kedua digunakan variabel:

```
unsigned long previousMillis2 = 0;
const long interval2 = 500;
bool ledState2 = false;
```

Variabel tersebut memiliki fungsi yang sama, tetapi interval LED kedua diatur sebesar 500 ms.

Pada fungsi `setup()`:

```
pinMode(13, OUTPUT);
pinMode(12, OUTPUT);
```

Pin 13 dan pin 12 dikonfigurasi sebagai output untuk LED.

Pada fungsi `loop()`, program mengambil waktu saat ini menggunakan:

```
unsigned long currentMillis = millis();
```

Kemudian dilakukan pengecekan untuk LED pertama:

```
if (currentMillis - previousMillis1 >= interval1)
```

Jika selisih waktu telah mencapai 1 detik, maka:

```
previousMillis1 = currentMillis;
```

digunakan untuk memperbarui waktu terakhir LED berubah.

Selanjutnya:

```
ledState1 = !ledState1;
```

digunakan untuk membalik kondisi LED dari ON ke OFF atau sebaliknya.

Kemudian kondisi LED diterapkan menggunakan:

```
digitalWrite(13, ledState1);
```

Proses yang sama dilakukan pada LED kedua, namun dengan interval 500 ms sehingga LED kedua berkedip lebih cepat dibanding LED pertama. Hasil percobaan menunjukkan bahwa kedua LED dapat berkedip dengan interval berbeda secara bersamaan tanpa menggunakan `delay()`. Hal ini membuktikan bahwa metode `millis()` dapat digunakan untuk multitasking sederhana pada Arduino.

4.3 ANALISIS PERTANYAAN PRAKTIKUM

4.3.1 KEUNTUNGAN INTERRUPT DIBANDING POLLING

Interrupt dan polling merupakan dua pendekatan berbeda dalam menangani event pada mikrokontroler. Berikut adalah perbandingan keunggulan interrupt terhadap polling:

Tabel 3.4 Perbandingan Interrupt dan Polling

Aspek	Interrupt	Polling
Respons	Sangat cepat, langsung saat event terjadi	Bergantung pada frekuensi loop pengecekan
Efisiensi CPU	Tinggi, CPU bebas saat tidak ada event	Rendah, CPU terus bekerja untuk memeriksa
Konsumsi Daya	Lebih hemat daya	Lebih boros daya
Kompleksitas Kode	Sedikit lebih kompleks (ISR, volatile)	Lebih sederhana

Multitasking	Mendukung multitasking lebih baik	Sulit untuk multitasking
--------------	-----------------------------------	--------------------------

Keuntungan utama interrupt adalah efisiensi penggunaan CPU. Ketika tidak ada event yang perlu ditangani, CPU dapat menjalankan tugas lain atau memasuki mode hemat daya. Menurut Saputra dan Budiman (2023), sistem berbasis interrupt jauh lebih efisien pada aplikasi yang memerlukan respons terhadap event asinkron yang tidak dapat diprediksi waktunya, seperti penekanan tombol oleh pengguna atau sinyal dari sensor eksternal [1].

Selain itu, respons interrupt lebih deterministik dan cepat. Pada sistem polling, event yang terjadi di antara dua siklus pengecekan bisa terlewat atau tertunda responsnya. Pada interrupt, event langsung ditangkap oleh hardware dan ISR dipanggil dalam hitungan siklus clock, memastikan tidak ada event yang terlewat selama interrupt tidak sedang dinonaktifkan [2].

4.3.2 PENTINGNYA TIMER DALAM SISTEM EMBEDDED DAN REAL-TIME

Timer merupakan komponen fundamental dalam sistem embedded karena hampir semua aplikasi embedded memerlukan pengelolaan waktu dengan presisi tertentu. Menurut Firmansyah dan Wibowo (2024), timer digunakan untuk menghasilkan sinyal PWM, mengukur durasi event, menjadwalkan task secara periodik, serta menentukan batas waktu (timeout) suatu operasi [3].

Dalam sistem real-time, ketepatan waktu adalah persyaratan utama. Sebuah sistem real-time yang gagal merespons dalam batas waktu yang ditentukan dianggap mengalami kegagalan fungsional, bahkan jika secara logika hasilnya benar. Timer memungkinkan sistem untuk menjamin respons dalam batas waktu yang telah ditentukan (deadline). Pada Arduino Uno, Timer0 digunakan oleh fungsi `millis()` dan `delay()`, Timer1 memiliki resolusi 16-bit untuk keperluan presisi tinggi, dan Timer2 digunakan untuk PWM dan nada suara.

4.3.3 ALUR KERJA SISTEM KOMBINASI INTERRUPT DAN TIMER

Ketika interrupt dan timer digabungkan dalam satu sistem, terbentuk arsitektur yang sangat powerful untuk menangani berbagai kondisi secara bersamaan. Berikut adalah alur kerja kombinasinya:

- Program utama berjalan pada `loop()` dan menggunakan `millis()` untuk mengelola task-task periodik seperti pembacaan sensor, pengiriman data, atau update tampilan.
- Ketika interrupt eksternal terjadi (misalnya tombol ditekan atau sensor aktif), eksekusi program utama dihentikan sementara oleh hardware.
- ISR yang telah didaftarkan dieksekusi dengan cepat, biasanya hanya melakukan perubahan flag atau penyimpanan data minimal.
- Setelah ISR selesai, program utama dilanjutkan dari titik yang sama persis sebelum interrupt terjadi, termasuk nilai `millis()` yang masih valid.
- Program utama kemudian memeriksa flag yang di-set oleh ISR dan melakukan pemrosesan yang lebih kompleks jika diperlukan.

Menurut Kurniawan dan Santoso (2021), kombinasi interrupt dan timer merupakan fondasi dari sebagian besar implementasi RTOS (Real-Time Operating System) ringan pada mikrokontroler, di mana timer interrupt digunakan sebagai context-switch scheduler dan external interrupt digunakan untuk menangani I/O asinkron [5]. Pola ini memastikan sistem dapat bersifat responsif (melalui interrupt) sekaligus teratur dalam pengelolaan waktu (melalui timer).

4.3.4 DAMPAK ISR YANG TERLALU PANJANG ATAU KOMPLEKS

ISR yang terlalu panjang atau kompleks dapat menyebabkan berbagai masalah serius pada sistem embedded, antara lain:

- Kehilangan interrupt (Missed Interrupts): Selama ISR berjalan, interrupt lain dengan prioritas sama atau lebih rendah tidak dapat diproses. Jika event interrupt datang

lebih cepat dari waktu ISR selesai, event tersebut bisa terlewat.

- Ketidakakuratan Timer: Fungsi `millis()` bergantung pada Timer0 interrupt. Jika ISR memblokir eksekusi terlalu lama, penghitungan waktu `millis()` dapat menjadi tidak akurat.
- Ketidakresponsifan Sistem: Program utama tidak dapat berjalan selama ISR aktif, sehingga semua task yang bergantung pada `loop()` akan tertunda.
- Stack Overflow: ISR menggunakan stack yang sama dengan program utama. ISR yang kompleks dengan banyak variabel lokal berpotensi menyebabkan stack overflow.
- Deadlock: Jika ISR mencoba mengakses sumber daya yang sedang dikunci oleh program utama, sistem dapat mengalami kondisi deadlock.

Menurut Hidayat dan Prasetyo (2022), praktik terbaik yang direkomendasikan adalah membuat ISR hanya melakukan operasi minimum seperti mengubah flag, menyimpan data ke buffer, atau menonaktifkan perangkat, kemudian membiarkan program utama melakukan pemrosesan lebih lanjut berdasarkan flag tersebut. Pendekatan ini dikenal sebagai deferred processing atau bottom-half handling [2]. Rahmawati dan Kusuma (2023) juga menegaskan bahwa ISR yang ideal memiliki durasi eksekusi dalam orde puluhan hingga ratusan siklus clock, bukan ribuan [4].

5. KESIMPULAN

Praktikum Modul 6 berhasil mengimplementasikan dua mekanisme penting dalam sistem embedded Arduino Uno, yaitu external interrupt dan timer non-blocking. Pada Percobaan 6A, sistem berhasil mengendalikan LED menggunakan mekanisme interrupt dengan push button pada pin 2 mode FALLING, dimana ISR `tombolInterrupt()` meng-toggle variabel volatile `ledState` tanpa memerlukan polling pada loop utama. Pada Percobaan 6B, LED berhasil dikedipkan setiap 1 detik menggunakan fungsi `millis()` sebagai pengganti `delay()`, membuktikan bahwa sistem dapat tetap responsif selama periode penghitungan waktu.

Dari analisis pertanyaan praktikum dapat disimpulkan bahwa interrupt lebih unggul dari

polling dalam hal efisiensi CPU, respons waktu, dan konsumsi daya; timer sangat krusial dalam sistem real-time sebagai dasar penjadwalan dan pengendalian waktu; kombinasi interrupt dan timer membentuk arsitektur sistem yang responsif sekaligus terstruktur; dan ISR yang terlalu kompleks dapat menyebabkan missed interrupt, ketidakakuratan timer, serta ketidakresponsifan sistem. Pemahaman mendalam tentang kedua mekanisme ini merupakan fondasi penting dalam pengembangan aplikasi IoT, robotika, dan sistem otomasi berbasis mikrokontroler.

DAFTAR PUSTAKA

- [1] A. Saputra dan D. Budiman, "Implementasi Mekanisme Interrupt pada Sistem Kendali Berbasis Arduino untuk Aplikasi IoT," *Jurnal Teknologi dan Sistem Komputer*, vol. 11, no. 2, pp. 85-94, 2023.
- [2] R. Hidayat dan B. Prasetyo, "Analisis Kinerja Interrupt Service Routine pada Mikrokontroler AVR dalam Sistem Monitoring Real-Time," *Jurnal Ilmu Komputer dan Informatika*, vol. 8, no. 1, pp. 23-31, 2022.
- [3] F. Firmansyah dan A. Wibowo, "Optimasi Pengelolaan Waktu pada Arduino menggunakan Non-Blocking Timer untuk Sistem Multitasking Kooperatif," *Jurnal Elektronika dan Telekomunikasi*, vol. 24, no. 1, pp. 12-20, 2024.
- [4] S. Rahmawati dan H. Kusuma, "Perbandingan Konsumsi Daya antara Metode Polling dan Interrupt pada Sistem Embedded Berbasis ATmega328P," *Jurnal Pengembangan Teknologi Informasi dan Ilmu Komputer*, vol. 7, no. 4, pp. 1845-1853, 2023.
- [5] A. Kurniawan dan T. Santoso, "Rancang Bangun Sistem Multitasking Kooperatif Berbasis Mikrokontroler menggunakan Mekanisme Interrupt dan Timer," *Jurnal Nasional Teknik Elektro dan Teknologi Informasi*, vol. 10, no. 3, pp. 201-210, 2021.